

Lecture 5

Sparse matrices

structured (usually formed by diagonals)
unstructured (irregular entries)

- There are many cases when our matrices are composed of mostly zeros $\rightarrow$ These matrices could benefit from "structuring" their data in a clever way.

Examples:

- The matrix A corresponding to analysis of purchases:

	Product 1	Product 2	...	Product j	...
Customer 1	a_{11}	a_{12}	...	a_{1j}	...
Customer 2	a_{21}	a_{22}	...	a_{2j}	...
$\vdots$	$\vdots$	$\vdots$		$\vdots$	
Customer i	a_{i1}	a_{i2}	...	a_{ij}	...
$\vdots$	$\vdots$	$\vdots$		$\vdots$	

$\uparrow$ Then a_{ij} is the amount of product j customer i has bought.

Then, customer i will obtain its bill by $A[j, i]$.

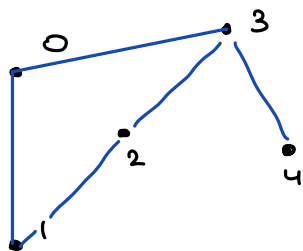
We can save $A[j, i]$ as a sparse array.

for storage and computer efficiency.

- Network analysis: We can store the information of whether two nodes in a network are connected in a matrix A .

$$A[j, k] = \begin{cases} 1 & \text{if node } j, k \text{ are connected} \\ 0 & \text{otherwise.} \end{cases}$$

E.g.



$$\Rightarrow A = \begin{pmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

Let's say we have a network of 10^4 nodes. Then the corresponding connection matrix will be of size $(10^4)(10^4) = 10^8$ but most of the entries will contain zeros.

Sparse representation only stores the non-zero values, i.e., the actual connections.

$\Rightarrow$ Moreover, A is symmetric $\Rightarrow$ only upper triangular part is enough to store.

How can we give the sparse representation of a matrix?

We store only:

1. nonzero values
2. Their positions
3. Information about where each row or column begins.

In Python, we use `csr-matrix` or `csc-matrix`
 $\uparrow$ compressed sparsed row.

both provided by the `scipy` library.

Alternatives: `lil-matrix` $\rightarrow$ For each row, it stores a list of column indices and a list of corresponding values.

This allows us to modify matrices in this format more easily since `A = lil_matrix((n,m))`

$\nearrow$ `A.rows`: list of column indices
 $\searrow$ `A.data`: values.

$\Rightarrow$ we can change or add entries in A normally: `A[i,j] = number`

Good news: we can multiply or even concatenate sparse matrix.
 Now we can multiply huge matrices that previously would have caused us some problem (memory overflow).

Remark: To check the density of a matrix A , we compute:

$$\text{density}(A) = \frac{\# \text{non-zeros}}{\# \text{total elements}}$$

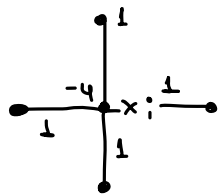
If this number is low $\Rightarrow A$ is sparse.

More examples

When we are discretizing numerical solution of PDEs we need sparse matrices. why?

$\hookrightarrow$ They have non-zero (main) diagonal elements

We can also see this problem as a network with nodes and edges.



$$A(i, i) = -4.$$

$\rightarrow$ Non zero subdiagonal and super diagonal elements:

(horizontal) x_i connects with both x_{i-1} and x_{i+1} neighbors

$$\begin{bmatrix} -4 & 1 & & & \\ 1 & -4 & 1 & & \\ & \ddots & \ddots & \ddots & \\ & & & 1 & -4 \end{bmatrix}$$

$\rightarrow$ A few more diagonal with non-zeros:

(vertical) x_i connects with x_{i+N} and x_{i-N} neighbors

$$\begin{bmatrix} -4 & 1 & & & & & \\ & 1 & -4 & & & & \\ & & \ddots & \ddots & \ddots & & \\ & & & & 1 & -4 & \\ & & & & & 1 & -4 \\ & & & & & & \ddots \end{bmatrix}$$

Kronecker product: Will allow us to construct structures with "repeated" patterns. Let $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{p \times q}$. Then,

$$A \otimes B = \begin{bmatrix} a_{11}B & a_{12}B & \dots & a_{1n}B \\ a_{21}B & & & \\ \vdots & & & \\ a_{m1}B & \dots & a_{mn}B \end{bmatrix} \in \mathbb{R}^{(p \cdot m) \times (q \cdot n)}$$

How can this be useful?

Suppose we want to find a matrix X s.t

$$A \times B = C$$

where $A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$, $B = \begin{bmatrix} 1 & 0 \\ 3 & 1 \end{bmatrix}$ and $C = \begin{bmatrix} 7 & 3 \\ 2 & 1 \end{bmatrix}$

Nice result:

$$\text{vec}(A \times B) = (B^T \otimes A) \text{vec}(X)$$

here $\text{vec}(\dots)$ stacks whatever is inside in columns, e.g. $\text{vec}\begin{pmatrix} x_1 & x_2 \\ x_3 & x_4 \end{pmatrix} = (x_1, x_2, x_3, x_4)^T$.

We solve equivalently $\text{vec}(A \times B) = \text{vec}(C)$ and thanks to $\uparrow$ we have:

$$\begin{pmatrix} 1 & 3 \\ 0 & 1 \end{pmatrix} (B^T \otimes A) \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 7 \\ 3 \\ 2 \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 3 & 6 \\ 0 & 1 & 0 & 3 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 7 \\ 3 \\ 2 \\ 1 \end{bmatrix}$$

Back to sparse matrices...

- The sparsity of a matrix can be improved by changing the numbering of the connectivity matrix. $\Rightarrow$ This can make linear solvers faster and more efficient.

Linear systems

$$A \in \mathbb{R}^{n \times m} \rightarrow Ax = b$$

$\uparrow$ $x \in \mathbb{R}^m$ $\uparrow$ $b \in \mathbb{R}^n$

Recall $Ax = x_1 A_1 + x_2 A_2 + \dots + x_m A_m \Rightarrow$ If $Ax = b$, then b can be written as a linear combination of

$$\text{Col}(A) := \{A_1, A_2, A_3, \dots, A_m\}$$

Then, we know that there exists a solution of $Ax = b$ iff $b \in \text{Col}(A)$.

An important theorem that will help us: $m = \dim(\text{Nul}(A)) + \text{Rank}(A)$
 $\uparrow$
rows (equations)

Let's consider 3 cases:

• $n > m$ (more equations than unknowns) $\rightarrow$ $b \in \text{Col}(A)$ $\rightarrow$ there exists $y \neq 0$ in $\text{Nul}(A)$: ∞ -solutions!
 $\rightarrow$ $\text{Nul}(A) = \{0\}$: only one solution!
 $\rightarrow b \notin \text{Col}(A) \rightarrow$ no solution.

• $n < m$ (more unknowns than equations) $\Rightarrow$ By the dimension theorem:

$$\text{Rank}(A) \leq n < m \Rightarrow \text{Null space is not empty!}$$

Hence, in this case either we have solutions, or there are ∞ -many of them.

• $n = m$ (same number of equations and unknowns). If we check at $\text{rank}(A)$ and we see $\text{rank}(A) < n$, then by the dimension theorem the null space is not empty and thus if a particular solution exists of $Ax = b$, then there are ∞ -many of them. This case is particularly nice!

What do we do when we can't expect to have solution?

Find x such that $Ax \approx b$ is our best approximation
 $\uparrow$
??? $\xrightarrow{\quad\quad\quad}$ $Ax - b$ should be minimal in some sense.
 $\downarrow$
To be determined

Back to our nice "square" case $n = m$. If this system is invertible, then the solution is unique! (i.e. $\text{Col}(A)$ spans all my space).

$$\begin{aligned} \hookrightarrow \text{Then } Ax &= b \\ \Leftrightarrow A^{-1}Ax &= A^{-1}b \\ x &= A^{-1}b \end{aligned}$$

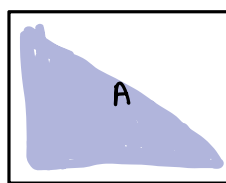
Theoretically fine but too impractical: A^{-1} is very expensive to compute.

$\Rightarrow$ We can study the algebraic structure of A . There are some nice cases:

Simplest case: When a simple forward substitution method works:

$$\begin{array}{lcl} x_1 = b_1 & & x_1 = b_1 \\ x_1 + x_2 = b_2 & \Rightarrow & x_2 = b_2 - x_1 \\ x_1 + x_2 + x_3 = b_3 & & x_3 = b_3 - x_2 - x_1 \\ \vdots & & \vdots \end{array}$$

In matrix form this system $Ax = b$ has only a lower triangular part



$$Ax = b$$

Can you write this method using a simple algorithm?

These type of methods are called direct methods

There are sophisticated methods that can be applied even for sparse matrices.

However, in most cases, we must rely in approximating our solution using some iterative method

$\hookrightarrow$ **Main idea:** find a sequence of vectors $\{x^{(k)}\}_{k=1}^{\infty}$

such that

$$\lim_{k \rightarrow \infty} x^{(k)} = x \quad \text{(in some sense).}$$

$\uparrow$
our solution



Main idea for the construction: Make $Ax = b$ into a fixed-point problem

$$u = Bu + c$$

for some invertible B and c .

Then, we can use a fixed-point iteration $u_{n+1} = Bu_n + c$.

$\Rightarrow u_n \rightarrow x^*$ the solution of $\boxed{Ax=b}$

How can we find B or C?

Strategy: let's see $Ax=b$ equation by equation:

$$x_1 = \frac{1}{a_{11}} \left(b_1 - \sum_{j \neq 1}^n a_{1j} x_j \right)$$

$a_{11} \neq 0, A \in \mathbb{R}^{n \times n}$

$$x_2 = \frac{1}{a_{22}} \left(b_2 - \sum_{j \neq 2}^n a_{2j} x_j \right)$$

$$\vdots$$

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i}^n a_{ij} x_j \right)$$

split $-A = \begin{bmatrix} & -F \\ -E & \end{bmatrix}; D = \text{diag}(A).$
 $\Rightarrow A = D - (E+F)$

let's use this as inspiration: $x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i}^n a_{ij} x_j^{(k)} \right)$

we make this into an iteration

This is called Jacobi iteration

$$x^{(k+1)} = D^{-1} (b + (E+F)x^{(k)})$$

This method converges if:

$$|A_{ii}| > \sum_{j \neq i} |A_{ij}| \quad (A \text{ is diagonally dominant})$$

$$= \underbrace{D^{-1}b}_C + \underbrace{D^{-1}(E+F)}_B x^{(k)}$$

The error of this method decreases as follows:

$$\begin{aligned} B &= D^{-1}(E+F) \\ &= D^{-1}(D-A) \\ &= I - D^{-1}A \end{aligned}$$

$$\frac{\|x - x^{n+1}\|}{\|x - x^n\|} \leq \rho < 1$$

$$\|x - x^{n+1}\| \leq \rho \|x - x^n\|$$

$$\leq \rho^n \|x - x^0\|$$

Linear convergence

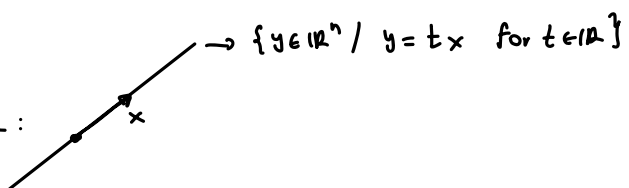
error decreases linearly,

lecture 6: The eigenvalue problem

Consider a square matrix $A \in \mathbb{R}^{n \times n}$.

Geometrical interpretation: Is it possible to find $x \in \mathbb{R}^n$ such that $Ax = \lambda x$ for some $\lambda \in \mathbb{R}$. In other words, can we find vectors x in $\mathbb{R}^n$ such that our transformation A only "scales" them (i.e. the subspace they generate remains "unchanged" by A).

- $x \neq 0$ is called an eigenvector and λ its respective eigenvalue.

Remark: Think of the subspace x generates: 

Then it makes sense that any element in this subspace is also an eigenvector.

→ it is sufficient to work with eigenvectors of length one.

Remark: If $x \neq 0$ is an eigenvector and $\lambda = 0$ is an eigenvalue, then A^{-1} can't exist, otherwise:

$$\begin{aligned} Ax &= 0 \cdot x \\ A^{-1}(Ax) &= A^{-1}(0 \cdot x) \\ x &= 0 \quad (\text{contradiction}) \end{aligned}$$

Remark: We can rewrite $Ax = \lambda x \Rightarrow (A - \lambda I)x = 0$. Then the above suggest that we can find eigenvalues by searching for λ s.t

$$\text{rank}(A - \lambda I) < n$$

(otherwise $(A - \lambda I)^{-1}$ exists and we arrive at a contradiction again).

Remark: If $x \neq 0$ is an eigenvector, then $(A - \lambda I)x = 0 \Rightarrow x \in \text{Null}(A - \lambda I)$

Remark: Looking for λ s.t $(A - \lambda I)^{-1}$ does not exist is equivalent to

finding λ s.t $\det(A - \lambda I) = 0$

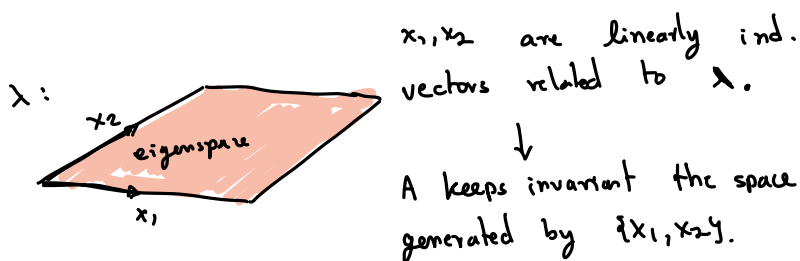
$p(\lambda)$ this is a polynomial with respect to λ , usually called the characteristic polynomial of A

Example: What are the eigenvalues of $I = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$? That is, we are looking for $\lambda \in \mathbb{R}$ s.t. $Ix = \lambda x$. Then, clearly $\lambda = 1$. What are its eigenvectors?

We can see that I does not change the whole space. Hence, the eigenvectors corresponding to $\lambda = 1$ are $\begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$, $\begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}$ and $\begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$.

We call $\lambda = 1$ a multiple eigenvalue.

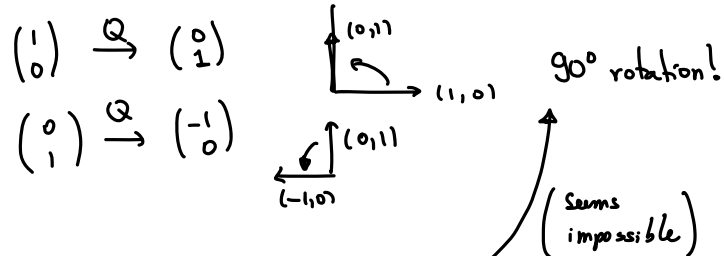
From the previous example, we see that one eigenvalue may have multiple linearly independent eigenvectors. Visually:



Notation: We sort the eigenvalues of A as follows: $|\lambda_1| > |\lambda_2| \geq |\lambda_3| \dots$ (decreasing order)

Some examples:

• $Q = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ Can you visualize what Q does?



In this case, can we find $\lambda, x \neq 0$ s.t. $Qx = \lambda x$?

$$\det(Q - \lambda I) = 0$$

$$\Leftrightarrow \det \begin{pmatrix} -\lambda & -1 \\ 1 & -\lambda \end{pmatrix} = 0$$

$$\Leftrightarrow \lambda^2 + 1 = 0 \quad \left. \begin{array}{l} \lambda_1 = +i \\ \lambda_2 = -i \end{array} \right\} \text{complex eigenvalues!}$$

Important result: If $A \in \mathbb{R}^{n \times n}$ is symmetric: all of its eigenvalues are real.

↳ their eigenvectors actually form an orthonormal basis of $\mathbb{R}^n$.

• Eigenvalues of a stochastic matrix

let's recall:

right stochastic matrix S : $\sum_j S[i, j] = 1$ for all i (row-wise)

left stochastic matrix S : $\sum_i S[i, j] = 1$ for all j (columnwise)

what are the eigenvalues if S is right stochastic?

Let us remember that we can sum the elements of each row of S

by multiplying S with a vector of ones:

$$S \begin{pmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix} = \begin{pmatrix} \sum_j S[1, j] \\ \sum_j S[2, j] \\ \vdots \\ \sum_j S[n, j] \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix}$$

↑ since S is right stochastic

- This implies that $(1 \dots 1)^T$ is an eigenvector with eigenvalue $\lambda = 1$.
 - Moreover, we can show that this is the largest eigenvalue: $|\lambda| \leq 1$ (for any e.value λ).
 - This is also valid for double stochastic matrices,
- but for left stochastic matrices finding eigenvector is no easy task.

An iterative method to find

dominant eigenvectors (and their eigenvalues)

↓

The power method

- Conditions: The first (dominant) eigenvalue should be unique, i.e.

$$|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \dots$$

- Initial guess may break the algorithm (we will see this with more details).

Algorithm: Choose $v_0 \in \mathbb{R}^n$ s.t. $\|v_0\| = 1$

Iterate: $v_{n+1} = \frac{Av_n}{\|Av_n\|}$ (can be done quickly if A is sparse)

why this works? Suppose we have n eigenvectors $x_1, \dots, x_n$. They form a basis in $\mathbb{R}^n$, i.e. we can write any vector in $\mathbb{R}^n$ as a linear comb. of $\{x_1, \dots, x_n\}$. Take our initial guess $v_0 \in \mathbb{R}^n$:

$$v_0 = c_1 x_1 + c_2 x_2 + \dots + c_n x_n \quad \text{for some numbers}$$

$$\text{Apply } A: \quad Av_0 = A(c_1 x_1 + \dots + c_n x_n)$$

$$= c_1 Ax_1 + \dots + c_n Ax_n$$

$$= c_1 \lambda_1 x_1 + \dots + c_n \lambda_n x_n$$

$$\text{Apply } A \text{ } k \text{ times!} \Rightarrow A^k v_0 = c_1 \lambda_1^k x_1 + \dots + c_n \lambda_n^k x_n$$

$$= \lambda_1^k \left(c_1 x_1 + \underbrace{c_2 \frac{\lambda_2^k}{\lambda_1^k} x_2 + \dots + c_n \frac{\lambda_n^k}{\lambda_1^k} x_n}_{\text{is very small}} \right)$$

If λ_1 is dominant, then $\frac{\lambda_j}{\lambda_1} < 1$ and $\frac{\lambda_j^k}{\lambda_1^k}$ is very small

$$\text{Then} \quad A^k v_0 \approx \lambda_1^k c_1 x_1$$

↑ If $c_1 = 0$, i.e.

 v_0 and x_1 are perpendicular (hence v_0 takes no components from x_1)

If we obtain x_1 (dominant eigenvector) from this method, how can we get its respective eigenvalue?

Then $A^k v_0 \approx 0$ and the algorithm breaks

⇓

v_0 should be a vector not orthogonal to the dominant eigenvector x_1 .

Answer: Rayleigh quotient: $\lambda = \frac{\langle Ax, x \rangle}{\|x\|^2}$

why? If v is an eigenvector of $A: Av = \lambda v$, then

$$\text{Rayl. quo.} = \frac{\langle Av, v \rangle}{\|v\|^2} = \frac{\lambda \|v\|^2}{\|v\|^2} = \lambda$$

However, the power method only approximates $x_1 \Rightarrow \lambda_1 \approx \frac{\langle Ax_1, x_1 \rangle}{\|x_1\|^2}$.

Convergence rate: $\left(\frac{|\lambda_2|}{|\lambda_1|}\right)^n$

↳ that is: $|x_1 - v_1| \sim \underbrace{\left(\frac{|\lambda_2|}{|\lambda_1|}\right)^n}_{\text{exponential approximation rate.}}$

↓

e.g. $\lambda_1 = 10, \lambda_2 = 5$:

$$\left(\frac{|\lambda_2|}{|\lambda_1|}\right) = 0.5$$

⇒ error would be something like:

n-th iteration	error factor
0	1
1	0.5
2	0.25
3	0.031
4	0.000977



$$0.5^n = \exp(-cn)$$

with $c = \ln(0.5)$